Programming Project 5

Graded

36 Minutes Late

Student
Jinku Jung

Total Points
60 / 70 pts

Autograder Score
60.0 / 70.0

Failed Tests
multiLetter - Good Input (0/10)

Passed Tests
Everything Compiles (30/30)
endsWith - Good Input (10/10)
lengthN - Good Input (10/10)
containsLetter - Good Input (10/10)

Question 2

Autograder Adjustment **0** / 0 pts

✔ **+ 0 pts** No adjustment made

Autograder Results

Everything Compiles (30/30)

endsWith - Good Input (10/10)

Tested with inputs: (z,4)

lengthN - Good Input (10/10)

Tested with input: (21)

multiLetter - Good Input (0/10)

Test Failed: expected:<[kickback, kickbacks]> but was:<[bangkok, bangkoks, buckskin, buckskins, kaka, kakap
at edu.columbia.pp5.WordListsTests.test_multiLetter_goodInput:61 (WordListsTests.java)
Tested with inputs: (3,k)

containsLetter - Good Input (10/10)

Tested with inputs: (q,1,5)

Submitted Files

dj2598_pp5/dictionary.txt Download

1	Large file hidden. You can download it using the button above.

dj2598_pp5/readMe.txt Download

```
README.txt
Name: Jinku Jung
UNI: dj2598
Assignment: Programming Project #5
Date: 4/12/2021

================================================================================
WordLists.java: File input and specific-case Scrabble word search methods.
================================================================================
Uses the Files method from java.nio.file.* package to quickly read all lines
from a given Scrabble dictionary file.  Once in the heap, the dictionary is
stored live as an ArrayList, "scrabble_Words."  Here, the four Scrabble Word
Search methods created also in this class can freely check their parameter
conditions against every word in the dictionary.  Various applicable
exceptions are thrown if inputs, calculations, or storage space go awry.

================================================================================
WordTest.java: Using WordLists' Scrabble word search methods-->output to *.txt
================================================================================
From the heap ArrayList structure, scrabbleWords, accessed via a WordLists
object reference variable 'w,' the four Scrabble Word Search methods are
tested for niche cases than would yield reasonably short but unique results,
still relevant for debugging purposes.  Various applicable
exceptions are thrown if inputs, calculations, or storage space go awry.
Plus, the main's methods are enclosed in a "try ... catch"  structure to
help explain any plausible errors due to reasonable runtime exceptions which
may being encountered through this program's usage.

--------------------------------------------------------------------------------

Final Thoughts: Big Java is somewhat useful but horribly terse when
                explaining the nuts and bolts of I/O and exception handling.
                Intro to CS should not be so "teach yourself how to code,"
                ideally.  But, remembering what real world IT Help Desk was
                like, we were expected to do this every day: consider it
                part of the job.  Sometimes you just have to hunker down
                "get 'er done!" <a la Larry the Cable Guy>.

                                                --Jinku Jung

--------------------------------------------------------------------------------
```

dj2598_pp5/WordTest.java

Download

```java
//************************************
//
//  WordTest.java
//
//  Test class for WordLists.java
//  Programming Project 5, COMS W1004
//
//
//  Your Name: Jinku Jung
//  Your Uni: dj2598
//************************************

//Cut & Pasted from VS Code on 4/10/2021 @ 7:43 PM EST  --Jinku

import java.util.ArrayList;
import java.io.PrintWriter;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.UnsupportedEncodingException;

public class WordTest {

   /* Main method: Tests my specific Scrabble word search methods for
                accuracy and precision in results returned and
                methods executed.
   */

   public static void main(String[] args)
   {
      WordLists w;

      try
      {
         w = new WordLists("dictionary.txt");

         check_LengthN(w);
         check_EndsWith(w);
         check_ContainsLetter(w);
         check_MultiLetter(w);

         System.out.println("=====================================");
         System.out.println("Thank you for using my Scrabble Word "
                        + "Tester!\n\n--Jinku Jung\n");
      }

      catch (SecurityException e)
      {
         System.out.println("ERROR: Write-access of the output file was "
         + "denied.  Please verify your file read/write privileges with "
```

```
50            + "your System Administrator.");
51
52            e.getMessage();
53            e.getCause();
54            e.printStackTrace();
55        }
56
57        catch (UnsupportedEncodingException e)
58        {
59            System.out.println("ERROR: An unsupported character set has "
60            + "been detected, preventing file writing activity.  Please "
61            + "verify your computer and operating system support the UTF-8 "
62            + "character set standard and try again.");
63
64            e.getMessage();
65            e.getCause();
66            e.printStackTrace();
67        }
68
69        catch (IllegalArgumentException e)
70        {
71            System.out.println("ERROR: Illegal string or data format "
72            + "detected.  Please verify your Scrabble dictionary is "
73            + "free of any non-letter word entries.  If this persists, "
74            + "call technical support or your local IT Help Desk.");
75
76            e.getMessage();
77            e.getCause();
78            e.printStackTrace();
79        }
80
81        catch (IllegalStateException e)
82        {
83            e.getMessage();
84            e.getCause();
85            e.printStackTrace();
86        }
87
88        catch(StackOverflowError e)
89        {
90            e.getMessage();
91            e.getCause();
92            e.printStackTrace();
93        }
94
95        catch (IOException e)
96        {
97            System.out.println("ERROR: Your output file could not "
98            + "be closed.  Please verify "
99            + "your destination file has properly been saved.");
100
101           e.getMessage();
```

```
102         e.getCause();
103         e.printStackTrace();
104      }
105
106      catch (NullPointerException e)
107      {
108         e.getMessage();
109         e.getCause();
110         e.printStackTrace();
111      }
112   }
113
114   //Sub-methods of Main:
115
116   /* Method: check_LengthN(): taking in just one input
117            (while checking for illegal argument,
118            illegal state, unsupported encoding, and
119            file not found exceptions), it retrieves
120            and outputs the results of "lengthN()"
121            from the WordLists class.  It
122            returns nothing.
123   */
124
125   public static void check_LengthN(WordLists w)
126   throws IllegalArgumentException, IllegalStateException,
127   UnsupportedEncodingException, FileNotFoundException
128   {
129      int wordsFound = 0;
130      ArrayList<String> result = new ArrayList<String>();
131      PrintWriter writer = new PrintWriter("lengthN.txt", "UTF-8");
132
133      System.out.println("====================================");
134      System.out.println("\nTesting the lengthN() method . . .\n");
135
136      result = w.lengthN(2);
137      wordsFound = w.getNWords();
138
139      checkHit_lengthN(writer, result, wordsFound);
140      checkMiss_LengthN(writer, result);
141   }
142
143   /* Method: checkHit_LengthN(): verifies a positive
144            result from its parent method, check_LengthN,
145            then outputs its results to the text file,
146            "lengthN.txt."  It returns nothing.
147   */
148
149   public static void checkHit_lengthN(PrintWriter writer,
150   ArrayList<String> result, int wordsFound)
151   {
152      if(!result.isEmpty())
153      {
```

```
154         for(String word : result)
155         {
156            writer.println(word);
157            System.out.println(word);
158         }
159 
160         writer.println("\n" + wordsFound
161         + " total words found with your given condition!\n");
162 
163         System.out.println("\n" + wordsFound
164         + " total words found with your given condition!\n");
165         System.out.println("Closing PrintWriter...\n");
166         System.out.println("File saved as: lengthN.txt\n");
167 
168         writer.close();
169      }
170   }
171 
172   /* Method:  checkMiss_LengthN(): verifies a negative
173            result from its parent method, check_LengthN,
174            then outputs its results to the text file,
175            "lengthN.txt."  It returns nothing.
176   */
177 
178   public static void checkMiss_LengthN(PrintWriter writer,
179   ArrayList<String> result)
180   {
181 
182      if(result.isEmpty())
183      {
184         writer.println("No such words of said length exist.\n");
185 
186         System.out.println("No such words of said length exist.\n");
187         System.out.println("Closing PrintWriter...\n");
188         System.out.println("File saved as: lengthN.txt\n");
189 
190         writer.close();
191      }
192   }
193 
194   /* Method: check_EndsWith(): taking in just one input
195            (while checking for illegal argument,
196            illegal state, unsupported encoding, and
197            file not found exceptions), it retrieves
198            and outputs the results of "endsWith()"
199            from the WordLists class.  It
200            returns nothing.
201   */
202 
203   public static void check_EndsWith(WordLists w)
204   throws IllegalArgumentException, IllegalStateException,
205   UnsupportedEncodingException, FileNotFoundException
```

```
206     {
207        int wordsFound = 0;
208        ArrayList<String> result = new ArrayList<String>();
209        PrintWriter writer = new PrintWriter("endsWith.txt",
210        "UTF-8");
211
212        System.out.println("====================================");
213        System.out.println("\nTesting the endsWith() method . . .\n");
214
215        result = w.endsWith('x', 2);
216        wordsFound = w.getNWords();
217
218        checkHit_EndsWith(writer, wordsFound, result);
219        checkMiss_EndsWith(writer, result);
220     }
221
222     /* Method: checkHit_EndsWidth(): verifies a positive result from
223              its parent method, check_EndsWith, then outputs its
224              results to the text file, "endsWith.txt."
225              It returns nothing.
226     */
227
228     public static void checkHit_EndsWith(PrintWriter writer,
229     int wordsFound, ArrayList<String> result)
230     {
231        if(!result.isEmpty())
232        {
233           for(String word : result)
234           {
235              writer.println(word);
236              System.out.println(word);
237           }
238
239           writer.println("\n" + wordsFound
240           + " total words found with your given conditions!\n");
241
242           System.out.println("\n" + wordsFound
243           + " total words found with your given conditions!\n");
244           System.out.println("Closing PrintWriter...\n");
245           System.out.println("File saved as: endsWith.txt\n");
246
247           writer.close();
248        }
249     }
250
251     /* Method: checkMiss_EndsWidth(): verifies a negative result from
252              its parent method, check_EndsWith, then outputs its
253              results to the text file, "endsWith.txt."
254              It returns nothing.
255     */
256
257     public static void checkMiss_EndsWith(PrintWriter writer,
```

```
258     ArrayList<String> result)
259     {
260        if(result.isEmpty())
261        {
262           writer.println("No such words end with that letter, "
263           + "per said word length or total letters.\n");
264
265           System.out.println("No such words end with that letter, "
266           + "per said word length of total letters.\n");
267           System.out.println("Closing PrintWriter...\n");
268           System.out.println("File saved as: endsWith.txt\n");
269
270           writer.close();
271        }
272     }
273
274     /* Method: check_ContainsLetter(): taking in just one input
275              (while checking for illegal argument, illegal state,
276              unsupported encoding, and file not found exceptions),
277              it retrieves and outputs the results of
278              "containsLetter()" from the WordLists class.  It
279              returns nothing.
280     */
281
282     public static void check_ContainsLetter(WordLists w)
283     throws IllegalArgumentException, IllegalStateException,
284     UnsupportedEncodingException, FileNotFoundException
285     {
286        int wordsFound = 0;
287        ArrayList<String> result = new ArrayList<String>();
288        PrintWriter writer = new PrintWriter("containsLetter.txt", "UTF-8");
289
290        System.out.println("====================================");
291        System.out.println("\nTesting the containsLetter() "
292                                  + "method . . .\n");
293
294        result = w.containsLetter('z', 1, 5);
295        wordsFound = w.getNWords();
296
297        checkHit_ContainsLetter(writer, result, wordsFound);
298        checkMiss_ContainsLetter(writer, result);
299     }
300
301
302     /* Method: checkHit_ContainsLetter(): verifies a positive
303              result from its parent method, check_ContainsLetter,
304              then outputs its results to the text file,
305              "containsLetter.txt."  It returns nothing.
306     */
307     public static void checkHit_ContainsLetter(PrintWriter writer,
308     ArrayList<String> result, int wordsFound)
309     {
```

```
310
311         if(!result.isEmpty())
312         {
313             for(String word : result)
314             {
315                 writer.println(word);
316                 System.out.println(word);
317             }
318
319             writer.println("\n" + wordsFound
320             + " total words found with your given conditions!\n");
321
322             System.out.println("\n" + wordsFound
323             + " total words found with your given conditions!\n");
324             System.out.println("Closing PrintWriter...\n");
325             System.out.println("File saved as: containsLetter.txt\n");
326
327             writer.close();
328         }
329     }
330
331     /* Method:  checkMiss_ContainsLetter(): verifies a negative
332               result from its parent method, check_ContainsLetter,
333               then outputs its results to the text file,
334               "containsLetter.txt."  It returns nothing.
335     */
336
337     public static void checkMiss_ContainsLetter(PrintWriter writer,
338     ArrayList<String> result)
339     {
340
341         if(result.isEmpty())
342         {
343             writer.println("No such have said letter at said letter "
344             + "position, per said word length.\n");
345
346             System.out.println("No such have said letter at said letter "
347             + "position, per said word length.\n");
348             System.out.println("Closing PrintWriter...\n");
349             System.out.println("File saved as: containsLetter.txt\n");
350
351             writer.close();
352         }
353     }
354
355     /* Method: check_MultiLetter(): taking in just one input
356               (while checking for illegal argument, illegal state,
357               unsupported encoding, and file not found exceptions),
358               it retrieves and outputs the results of "multiLetter()"
359               from the WordLists class.  It returns nothing.
360     */
361
```

```
362    public static void check_MultiLetter(WordLists w) throws
363    IllegalArgumentException, IllegalStateException,
364    UnsupportedEncodingException, FileNotFoundException
365    {
366       int wordsFound = 0;
367       ArrayList<String> result = new ArrayList<String>();
368       PrintWriter writer = new PrintWriter("multiLetter.txt", "UTF-8");
369
370       System.out.println("======================================");
371       System.out.println("\nTesting the multiLetter() "
372                                  + "method . . .\n");
373
374       result = w.multiLetter(2, 'z');
375       wordsFound = w.getNWords();
376
377       checkHit_MultiLetter(writer, result, wordsFound);
378       checkMiss_MultiLetter(writer, result);
379    }
380
381    /* Method: checkHit_MultiLetter(): verifies a positive result from
382             its parent method, check_MultiLetter, then outputs its
383             results to the text file "multiLetter.txt."
384             It returns nothing.
385    */
386
387    public static void checkHit_MultiLetter(PrintWriter writer,
388    ArrayList<String> result, int wordsFound)
389    {
390       if(!result.isEmpty())
391       {
392          for(String word : result)
393          {
394             writer.println(word);
395             System.out.println(word);
396          }
397
398          writer.println("\n" + wordsFound
399          + " total words found with your given conditions!\n");
400
401          System.out.println("\n" + wordsFound
402          + " total words found with your given conditions!\n");
403          System.out.println("Closing PrintWriter...\n");
404          System.out.println("File saved as: multiLetter.txt\n");
405
406          writer.close();
407       }
408    }
409
410    /* Method:  checkMiss_MultiLetter(): verifies a negative
411             result from its parent method, check_MultiLetter,
412             then outputs its results to the text file,
413             "multiLetter.txt."  It returns nothing.
```

```
414     */
415
416     public static void checkMiss_MultiLetter(PrintWriter writer,
417     ArrayList<String> result)
418     {
419        if(result.isEmpty())
420        {
421           writer.println("No such words have said letter, per "
422           + "said number of occurrences.\n");
423
424           System.out.println("No such words have said letter, per "
425           + "said number of occurrences.\n");
426           System.out.println("Closing PrintWriter...\n");
427           System.out.println("File saved as: multiLetter.txt\n");
428
429           writer.close();
430        }
431     }
432  }
```

dj2598_pp5/WordLists.java

Download

```java
//************************************
//
//  WordLists.java
//
//  Class to aid with Scrabble
//  Programming Project 5, COMS W1004
//
//
//  Your Name: Jinku Jung
//  Your Uni: dj2598
//************************************

//Cut & Pasted from VS Code on 4/10/2021 @ 7:43 PM EST  --Jinku

import java.util.ArrayList;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;

public class WordLists {

   /* Instance Variables: Simply declares the ArrayLists of the specifc
                      Scrabble word methods and their counter variable,
                      integer "nWords."
   */

   private ArrayList<String> scrabble_Words;
   private ArrayList<String> lengthN_Words;
   private ArrayList<String> endsWith_Words;
   private ArrayList<String> containsLetter_Words;
   private ArrayList<String> multiLetter_Words;
   private int nWords;

   /* Constructor: Simply instantiates the ArrayLists of the specifc
                  Scrabble word methods and their counter variable,
                  integer "nWords."
   */

   public WordLists(String fileName) throws SecurityException,
   StackOverflowError, FileNotFoundException, IOException,
   NullPointerException, IllegalArgumentException
   {

      lengthN_Words = new ArrayList<String>();
      endsWith_Words = new ArrayList<String>();
      containsLetter_Words = new ArrayList<String>();
      multiLetter_Words = new ArrayList<String>();

```

```
50      scrabble_Words = (ArrayList<String>)
51      Files.readAllLines(Paths.get(fileName),
52      StandardCharsets.UTF_8);
53    }
54
55    /*Methods: lengthN(): taking in one input (while checking
56            for illegal argument and null pointer exceptions),
57            it finds every word of length "n."  It returns an
58            ArrayList of Strings.
59    */
60
61    public ArrayList<String> lengthN(int n)
62    throws IllegalArgumentException, NullPointerException
63    {
64      int length = 0;
65      nWords = 0;
66
67      for(String word : scrabble_Words)
68      {
69        length = word.length();
70
71        if(length == n)
72        {
73          lengthN_Words.add(word);
74          nWords++;
75        }
76      }
77      return lengthN_Words;
78    }
79
80    /* Method: endsWith(): taking two inputs, (while checking
81            for illegal argument and null pointer exceptions),
82            it finds all words of length "n" that end with
83            character "lastLetter."  It returns
84            an ArrayList of Strings.
85    */
86    public ArrayList<String> endsWith(char lastLetter, int n)
87    throws IllegalArgumentException, NullPointerException
88    {
89
90      int length = 0;
91      char myLast = '\0';
92
93      nWords = 0;
94
95      for(String word : scrabble_Words)
96      {
97        length = word.length();
98        myLast = word.charAt(length - 1);
99
100       if( (n == length) && (lastLetter == myLast) )
101       {
```

```
            endsWith_Words.add(word);
            nWords++;
         }
      }
      return endsWith_Words;
   }

   /* Method: containsLetter(): taking in three inputs
            (while checking for illegal argument and
            null pointer exceptions), it finds all
            words of length "n" which have the letter
            "included" exactly and the array/string
            position of element "index."  It
            returns an ArrayList of Strings.

   */

   public ArrayList<String> containsLetter(char included,
   int index, int n) throws IllegalArgumentException,
   NullPointerException
   {
      int length = 0;
      char myLetter = '\0';

      nWords = 0;

      for(String word : scrabble_Words)
      {

         length = word.length();
         myLetter = word.charAt(index);

         if( (included == myLetter) && (length == n) )
         {
            containsLetter_Words.add(word);
            nWords++;
         }
      }
      return containsLetter_Words;
   }

   /* Method: multiLetter() taking in two inputs (while
            checking for illegal argument and null
            pointer exceptions), it finds all words
            that have letter "included" present "m" times
            total throughout the whole word.  It returns
            an ArrayList of Strings.
   */

   public ArrayList<String> multiLetter(int m, char included)
   throws IllegalArgumentException, NullPointerException
   {
```

```
154         int length = 0;
155 
156         nWords = 0;
157 
158         for(String word : scrabble_Words)
159         {
160            int hits = 0;
161            length = word.length();
162 
163            if(word.indexOf(included) > -1)
164            {
165               int firstHit = word.indexOf(included);
166 
167               for(int j = firstHit; j < length; j++)
168               {
169                  if(word.indexOf(included, j) > -1)
170                  {
171                     hits++;
172                  }
173               }
174 
175               if(hits == m)
176               {
177                  multiLetter_Words.add(word);
178                  nWords++;
179               }
180            }
181         }
182         return multiLetter_Words;
183      }
184 
185      /* Method: getWords: taking in zero inputs, it just
186               returns the number of words found by a
187               specific Scrabble word search method.
188               It returns an integer value.
189      */
190 
191      public int getNWords() throws NullPointerException
192      {
193         return nWords;
194      }
195   }
```